SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Student Name	K.Pavithra	**Register Number**	192472155
Department / Section	Information Technology	**Academic Year / Semester**	2025-2026
Faculty Name	Dr.P.Murali	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Grey Level Transformation Analysis	4	3.5	Three transformations on 10 pages; contrast 39.5 → 69.9 with stretching, curves, histograms and bit-plane analysis; γ and pencil-mark what-ifs measured. Pretrained enhancement model reasoned, not run.
2	Histogram Processing and Enhancement	4	3.5	HE, matching and CLAHE compared on light, normal and shadowed pages with histograms/CDFs; local statistics explain the shadow; CLAHE clip/tile sweep justified. Deep model comparison not run.
3	Spatial Filtering and Image Smoothing	5	4.5	Box, Gaussian and median at 3/5/7 kernels with PSNR, SSIM, Dice and profiles; correlation vs convolution checked by hand against filter2D. DnCNN / diffusion reasoned only.
4	Image Sharpening and Edge Enhancement	5	4.5	Laplacian, unsharp/high-boost and gradient sharpening with variance of Laplacian, blank-paper noise, halos and OCR CER; smooth-then-sharpen justified. Deep deblurring reasoned only.
5	Frequency-Domain Low-Pass Filtering	5	5	Spectrum shows the stripe peaks; ideal / Butterworth / Gaussian LPF with ringing and Tesseract CER (61.6 % → 1.9 %); Gaussian equivalence and padding wrap-around measured. Excellent.

6	Frequency-Domain High-Pass Filtering	4	3.5	High-pass, high-frequency emphasis and homomorphic filtering on shadowed pages; contrast 18 → 41 with DC preserved; ideal-HPF consequences explained. Learned shadow removal not run.
7	Point, Line, and Edge Detection	5	4.5	Point, line and edge detectors scored with precision / recall per structure (2 px tolerance); thresholds T and Canny justified; tilt failure of line masks explained. HED not run.
8	Image Thresholding and Binarization	4	3.5	Otsu, global and adaptive thresholds on 10 pages with Dice (0.84 adaptive), histograms with T and f = i·r explanation; block-size sensitivity measured. DIBCO model reasoned only.
9	Region-Based Segmentation and Integrated Pipeline Design	6	5	Region growing, watershed and edge linking with stain IoU 0.89 and corner error 2.1 px; integrated pipeline 1.46 s with stage times, block diagram and decision table. SAM 2 not run.
10	Smart Scanner Adaptation for High-Resolution Phone Photographs	8	7.5	Impact analysis and audit lead to MODIFY: Dice 0.79, blue notes kept, 119 ms per page. Phone photos are generated rather than captured; LayoutLMv3 / Donut discussed but not run.
	TOTAL	**50**	**45**	**Percentage: 90 %**

OVERALL REMARKS

Very good work: every task applies the three techniques on the same pages, measures them with suitable metrics (contrast, PSNR/SSIM, Dice, IoU, Tesseract CER, time) and ends with a justified decision. The frequency-domain and segmentation tasks are particularly strong, and the integrated pipeline is timed stage by stage. Marks are held back because all pages and phone photos are generated rather than real scans, and the pretrained / deep-learning comparisons (DocEnTr, DnCNN, NAFNet, BEDSR-Net, HED, DIBCO U-Net, SAM 2, LayoutLMv3) are reasoned with code but not executed. In the viva, ask the student to run the final pipeline on a few real library pages and explain one deep-learning comparison.

K.Pavithra		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Note
1	Transformation curves, before/after pages and histograms, a contrast table for the three techniques, and a justified choice with parameters.	✓	Curves, histograms and contrast table present

2	Histogram/CDF plots, before/after pages, a contrast/entropy table for the three techniques, and justified parameter choices.	✓	Histograms, CDFs and CLAHE sweep present
3	Filtered pages, a PSNR/SSIM/time table, text profiles, and a justified filter and kernel size.	✓	PSNR/SSIM tables, profiles and hand calculation
4	Sharpened pages, stroke profiles, a sharpness/noise table for the three filters, and a justified filter with parameters.	✓	Profiles, sharpness/noise table and OCR results
5	Spectra, filtered pages for three cut-offs, a ringing/blur/time table for the three filters, and a justified filter and D0.	✓	Spectrum, ringing table and OCR before/after
6	High-pass outputs, a contrast/evenness table for the three approaches, and a justified choice with parameters.	P	Measured tables present; learned shadow removal not run
7	Detection maps, a precision/recall table for points, lines and edges, and a justified detector for each item.	✓	Detection images and precision/recall per structure
8	Binary pages, annotated histograms, a Dice table for the three techniques, and a justified thresholding method.	P	Dice table and histograms; deep binarization not run
9	Segmentation results and a comparison table for the three approaches, an original block diagram with timings, a decision table, and a defended final pipeline.	✓	IoU/corner table, block diagram and decision table
10	Three adaptation strategies applied and compared with measured evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.	P	Strategies measured on generated phone photos

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Source code (Python/OpenCV, commented)	✓	Commented code for every technique; page generator shared by all tasks
Test inputs (pages, phone photos, ground-truth masks)	P	Generated pages with exact ground truth; no real scanned pages
Expected behaviour (predicted contrast, ringing, Dice)	✓	Predictions stated and checked against measurements (Task 9)
Actual output (measured contrast, PSNR, Dice, CER, time)	✓	Measured tables in every task, incl. Tesseract OCR
Intermediate evidence (histograms, spectra, masks)	✓	Histograms, spectra, profiles and masks included
Boundary / failure tests (shadows, stains, tilt, noise)	✓	Shadowed, stained, tilted and noisy pages tested
Interpretation of results	✓	Each table explained; contradicted predictions discussed
Code, data and report consistent	P	Consistent; deep-learning items given as code, not executed

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No
Three techniques applied and compared in every task	☒ Yes ☐ Partly ☐ No
Commented Python/OpenCV source code for every experiment	☒ Yes ☐ Partly ☐ No
Results obtained from own code execution or measurements (deep-learning items not executed)	☐ Yes ☒ Partly ☐ No
Same data and conditions used for all three techniques in a task	☒ Yes ☐ Partly ☐ No
Calculations and measurements shown with units	☒ Yes ☐ Partly ☐ No
Tables and graphs for experimental results	☒ Yes ☐ Partly ☐ No
Figures / screen outputs included in the submission	☒ Yes ☐ Partly ☐ No
Justified final decision with limits stated in every task	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended (run the pipeline on real library pages; explain one deep-learning comparison)	☒ Yes ☐ No

COMPUTER VISION
ASSIGNMENT 2 – CO2

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library (Total Marks – 50)

Scenario – Smart Document Scanner for the College Library

The college library wants to digitize its old printed question papers and lecture notes. The pages are yellowed and faded, some have tea stains and ink blots, and many were photographed with a mobile phone, so one side is darker because of a shadow. The library wants a simple "smart scanner" program (like the scanner apps on your phone) that (1) enhances each page so the text is clear and easy to read, and (2) segments the page: separates the text from the paper, finds the page border so the page can be cropped, finds ruled lines and table lines, and marks dust specks, punch holes and stains so library staff can check them.

You are the vision engineer. In every task, three candidate techniques are given. You must apply all three, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

ABSTRACT

Traffic sign recognition is an important component of intelligent transportation systems and smart city applications. Traffic signs provide essential information to drivers, including speed limits, road directions, restrictions, warnings, and pedestrian crossing instructions. Failure to recognize these signs may lead to traffic violations, accidents, and unsafe driving conditions. Therefore, an automated traffic sign detection system is required to identify traffic signs accurately and reliably under different environmental conditions.

The Smart Traffic Sign Detection System uses computer vision and image processing techniques to detect, enhance, segment, and recognize traffic signs captured by road cameras. The system is designed to work with images affected by sunlight, shadows, rain, night-time illumination, faded signs, partial occlusion, and motion blur.

The system begins with image acquisition from traffic cameras. The captured images are preprocessed to improve visual quality and reduce noise. Grey-level transformations and histogram processing are used to improve contrast and visibility. Spatial and frequency-domain filters are applied to reduce noise while preserving important sign boundaries. Sharpening techniques improve the visibility of edges and small details. Point, line, and edge detection methods identify significant structures in the image. Thresholding and region segmentation

separate traffic signs from the background. Finally, the extracted regions are passed to a traffic sign recognition model.

The project evaluates three candidate techniques for every major image-processing task. Their advantages, limitations, expected performance, and suitability are compared. The most appropriate technique is selected based on the requirements of traffic sign detection. Performance can be evaluated using image quality metrics, segmentation metrics, recognition accuracy, precision, recall, F1-score, intersection over union, and processing time.

The proposed system follows a hybrid approach that combines traditional image processing with a trained traffic sign detector. Traditional methods improve image quality and support region extraction, while the trained detector performs final sign recognition. This combination provides a flexible framework for smart city traffic monitoring and intelligent transportation applications.

Keywords: Traffic Sign Detection, Computer Vision, Image Processing, Smart City, Image Enhancement, Segmentation, Edge Detection, Object Recognition, Intelligent Transportation System.

CHAPTER 1: INTRODUCTION

1.1 Introduction to Computer Vision

Computer Vision is a field of artificial intelligence that enables computers to obtain meaningful information from digital images and videos. It allows machines to identify objects, recognize patterns, understand scenes, and make decisions based on visual information.

In traffic management, computer vision is used to analyze road scenes captured by cameras. It can identify vehicles, pedestrians, road markings, traffic signals, and traffic signs. The information obtained from these images can support traffic monitoring, driver assistance, road safety, and automated transportation systems.

A traffic sign detection system is designed to locate traffic signs in an image and identify their meaning. For example, the system may detect a STOP sign, a speed limit sign, a No Entry sign, or a pedestrian crossing sign. The detected sign can then be used to provide warnings or support traffic monitoring.

1.2 Importance of Traffic Signs

Traffic signs are essential for maintaining safety and order on roads. They communicate rules, warnings, and guidance to drivers and pedestrians.

The major categories of traffic signs include:

1. Regulatory signs: These signs communicate traffic rules and restrictions. Examples include STOP, No Entry, and speed limit signs.
2. Warning signs: These signs alert road users about possible hazards, such as sharp turns, road work, or pedestrian crossings.
3. Informational signs: These signs provide directions, distances, and other useful information.
4. Temporary signs: These signs are used during road construction, maintenance, or temporary traffic changes.

Recognizing these signs automatically can improve road safety and reduce the need for continuous manual monitoring.

1.3 Need for a Smart Traffic Sign Detection System

Traditional traffic sign recognition depends on human observation. Drivers must continuously observe the road and interpret signs while controlling their vehicles. This process can become difficult in poor weather, low-light conditions, or complex traffic environments.

A smart traffic sign detection system can automatically analyze images and identify signs. It can support:

- Intelligent transportation systems
- Advanced driver assistance systems

- Smart city traffic monitoring
- Automated traffic rule monitoring
- Road safety applications
- Driver warning systems

The system must be capable of recognizing signs even when the images are affected by environmental and camera-related problems.

1.4 Challenges in Traffic Sign Detection

Traffic sign detection is difficult because road images are captured under changing conditions.

1.4.1 Sunlight

Strong sunlight may produce overexposed regions. Bright areas can reduce the contrast between a traffic sign and its background. Details inside the sign may become difficult to recognize.

1.4.2 Night-Time Conditions

At night, traffic signs may receive limited illumination. The captured image may contain dark regions, sensor noise, and low contrast. Some signs may be visible only because of vehicle headlights or reflective materials.

1.4.3 Rain

Rain can create streaks, droplets, reflections, and blurred regions. These effects may introduce noise and reduce the visibility of sign boundaries.

1.4.4 Faded Signs

Traffic signs may become faded due to weather exposure and aging. Their colors and printed symbols may lose intensity, making recognition difficult.

1.4.5 Partial Occlusion

Trees, vehicles, poles, buildings, and other objects may partially cover a traffic sign. The system must identify the visible portion and avoid confusing it with another object.

1.4.6 Shadows

Shadows from buildings, trees, and vehicles may create uneven illumination. A sign may appear darker on one side and brighter on another side.

1.4.7 Motion Blur

When a camera or vehicle moves during image capture, the image may become blurred. Motion blur reduces the sharpness of sign boundaries and internal symbols.

1.5 Role of Image Processing

Image processing improves the quality of captured images before recognition. It includes operations such as:

- Noise reduction
- Contrast enhancement
- Brightness correction
- Edge enhancement
- Image filtering
- Thresholding
- Segmentation
- Boundary detection

These operations help prepare the image for reliable traffic sign recognition.

1.6 Role of Artificial Intelligence

Artificial intelligence allows a system to learn patterns from training data. A trained object detector can recognize traffic signs based on their shape, color, texture, and internal symbols.

Traditional image processing is useful for enhancing and analyzing images. However, it may not be sufficient for recognizing signs with complex backgrounds or partial occlusion. A trained detector can learn more complex visual patterns and improve recognition performance.

1.7 Project Overview

The proposed system contains the following major stages:

1. Image acquisition
2. Image preprocessing
3. Image enhancement
4. Noise reduction
5. Sharpening
6. Frequency-domain filtering
7. Point, line, and edge detection
8. Thresholding
9. Region segmentation
10. Traffic sign recognition
11. Result visualization
12. Performance evaluation

The system evaluates different techniques at each stage and selects the most suitable technique based on image quality, accuracy, computational cost, and robustness.

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

Traffic cameras capture road images under different environmental conditions. These images may contain traffic signs that are difficult to recognize because of poor illumination, rain, shadows, faded colors, partial occlusion, and motion blur.

The main problem is to develop an image-processing and computer-vision system that can enhance captured road images, identify important regions, detect traffic sign boundaries, separate traffic signs from the background, and produce recognition-ready image regions.

The system must compare multiple candidate techniques for each processing task and select the most suitable method based on measurable performance.

2.2 Existing System

In a traditional traffic sign monitoring system, traffic signs are mainly observed by human drivers or manually reviewed by traffic authorities.

Some existing automated systems use fixed color thresholds and shape-based rules to detect traffic signs. These systems may perform well under controlled lighting conditions but may fail when the sign color changes due to sunlight, shadows, fading, or night-time illumination.

Limitations of the Existing System

1. Manual observation requires continuous attention.
2. Fixed thresholding is sensitive to illumination changes.
3. Traditional shape-based methods may fail when signs are partially occluded.
4. Noise and blur reduce recognition accuracy.
5. Some methods require carefully selected parameters for different environments.
6. The system may produce false detections from objects with similar colors or shapes.
7. Recognition performance may decrease when traffic signs are small or distant.

2.3 Proposed System

The proposed Smart Traffic Sign Detection System combines image processing and trained object recognition.

The system captures road images and applies enhancement and filtering methods to improve image quality. It then identifies important edges and regions and prepares candidate traffic sign regions for recognition.

Main Features

1. Enhances images affected by poor lighting.
2. Reduces noise caused by rain and camera sensors.

3. Improves the visibility of faded traffic signs.
4. Detects sign boundaries and important visual regions.
5. Separates traffic signs from complex road backgrounds.
6. Uses a trained detector for final recognition.
7. Evaluates performance using standard metrics.
8. Displays detected signs and recognition results.

2.4 Objectives

The main objectives of the project are:

1. To acquire road images from traffic cameras.
2. To improve image quality under different lighting conditions.
3. To reduce noise while preserving sign boundaries.
4. To identify important edges and visual structures.
5. To segment traffic signs from the background.
6. To detect damaged, faded, or partially hidden signs.
7. To recognize traffic sign categories accurately.
8. To compare candidate image-processing techniques.
9. To evaluate system performance using measurable metrics.
10. To develop a recognition-ready output for smart city applications.

2.5 Scope of the Project

The project focuses on traffic sign detection from road images. It includes image preprocessing, enhancement, filtering, edge detection, thresholding, segmentation, recognition, and result visualization.

The system may be extended to video processing, real-time traffic monitoring, vehicle warning systems, and smart city traffic management.

2.6 Assumptions

The following assumptions are considered:

1. The input is a digital road image or video frame.
2. The traffic signs belong to known categories.
3. The system has access to a suitable traffic sign dataset for training and testing.
4. The image-processing parameters can be adjusted according to the camera and environment.

5. The trained detector is evaluated using data that is separate from its training data.
6. The expected performance values are hypotheses until verified through experiments.
7. Real-time performance depends on the camera resolution, hardware, and selected algorithms.

CHAPTER 3: SYSTEM REQUIREMENTS AND METHODOLOGY

3.1 Hardware Requirements

The system can be implemented using the following hardware:

- Computer or laptop
- Road camera or image dataset
- Processor
- RAM
- Optional GPU for deep learning
- Storage device

A GPU may improve the speed of model training and inference. However, basic image processing can be performed using a standard computer.

3.2 Software Requirements

The software requirements may include:

- Python
- OpenCV
- NumPy
- Matplotlib
- Scikit-image
- TensorFlow or PyTorch
- Jupyter Notebook or Visual Studio Code

OpenCV is used for image processing and computer vision operations. NumPy is used for numerical calculations. Matplotlib is used to visualize images and processing results. A deep learning framework can be used to train and evaluate the traffic sign detector.

3.3 General Methodology

The system follows a sequence of processing stages.

Stage 1: Image Acquisition

Road images are captured using a camera or collected from a traffic sign dataset.

Stage 2: Preprocessing

The image is resized, converted into a suitable color space, and prepared for processing.

Stage 3: Enhancement

Contrast and brightness are adjusted to improve the visibility of traffic signs.

Stage 4: Noise Reduction

Filtering techniques reduce noise while preserving important boundaries.

Stage 5: Feature Enhancement

Sharpening methods improve the visibility of sign edges and internal symbols.

Stage 6: Edge and Boundary Detection

Point, line, and edge detection methods identify significant structures.

Stage 7: Thresholding

Thresholding separates foreground regions from the background.

Stage 8: Segmentation

Segmentation identifies candidate traffic sign regions.

Stage 9: Recognition

A trained detector identifies the sign category.

Stage 10: Result Visualization

The detected signs are displayed with bounding boxes, labels, and confidence scores.

CHAPTER 4: TASK 1 — GREY-LEVEL TRANSFORMATIONS

4.1 Introduction

Grey-level transformation is an image enhancement technique that changes the intensity value of each pixel to improve the visual quality of an image.

A grayscale image contains intensity values ranging from 0 to 255 for an 8-bit image. A value of 0 represents black, while 255 represents white. Intermediate values represent different shades of grey.

In traffic sign detection, grey-level transformations are used to improve visibility when traffic signs are too dark, too bright, or have low contrast.

The transformation can be represented as:

$$s=T(r)$$

where:

- r is the input intensity
- s is the output intensity
- T is the transformation function

4.2 Candidate Technique 1: Image Negative Transformation

The negative transformation reverses the intensity values of an image.

$$s=L-1-r$$

For an 8-bit image:

$$s=255-r$$

Working Principle

Dark pixels become bright, and bright pixels become dark. This transformation may make certain details easier to observe when they are hidden in dark regions.

Advantages

1. Simple and easy to implement.
2. Improves visibility of some dark details.
3. Requires very little computation.
4. Can be applied to grayscale images.
5. Useful for visual inspection.

Limitations

1. It reverses the natural appearance of the image.
2. It may make traffic sign colors difficult to interpret.
3. It does not directly solve uneven illumination.
4. It may reduce the usefulness of color-based detection.
5. It is not suitable as the primary enhancement method for all traffic images.

Suitability

Negative transformation may be used for analysis or inspection, but it is not selected as the main method because traffic sign recognition depends on the natural appearance of sign colors and shapes.

4.3 Candidate Technique 2: Gamma Correction

Gamma correction adjusts the brightness of an image using a nonlinear transformation.

$$ s=c r^\gamma $$

For normalized image intensity:

$$ 0 \leq r \leq 1 $$

where:

- c is a constant
- γ is the gamma value

When $\gamma<1$, dark regions become brighter. When $\gamma>1$, the image becomes darker.

Working Principle

Gamma correction changes the intensity distribution without applying the same adjustment to every pixel.

Advantages

1. Useful for low-light images.
2. Can improve visibility of dark traffic signs.
3. Simple to implement.
4. Preserves the general structure of the image.
5. Can be adjusted according to lighting conditions.

Limitations

1. Incorrect gamma values may over-brighten the image.
2. Bright regions may lose detail.

3. It may amplify noise in dark regions.
4. It does not correct all forms of uneven illumination.
5. It requires parameter selection.

Suitability

Gamma correction is useful for night-time images and underexposed road scenes. However, it is not sufficient for images containing both very bright and very dark regions.

4.4 Candidate Technique 3: Piecewise-Linear Contrast Stretching

Piecewise-linear contrast stretching expands the intensity range of an image.

A simple transformation is:

$$ s= \begin{cases} 0, & r<r_1\\ \frac{(r-r_1)(s_2-s_1)}{r_2-r_1}+s_1, & r_1\leq r\leq r_2\\ 255, & r>r_2 \end{cases} $$

The intensity range of interest is expanded to improve contrast.

Working Principle

The lower and upper intensity limits are selected. Pixel values within this range are expanded, while values outside the range may be compressed or clipped.

Advantages

1. Improves contrast in low-contrast images.
2. Enhances faded sign details.
3. Simple to understand and implement.
4. Allows control over the intensity range.
5. Can be adjusted for different image conditions.

Limitations

1. Incorrect limits may remove important details.
2. It may amplify noise.
3. It does not always handle local illumination changes.
4. Bright and dark regions may require different parameters.
5. It may produce over-enhancement.

4.5 Comparison of Candidate Techniques

Technique	Main Purpose	Advantages	Limitations
Negative transformation	Reverses intensity	Simple; reveals some dark details	Changes natural appearance

Gamma correction	Brightness adjustment	Useful for dark images	May amplify noise
Contrast stretching	Expands intensity range	Improves faded and low-contrast details	Sensitive to selected limits

4.6 Selected Technique

Piecewise-linear contrast stretching is selected as the primary method.

It improves the contrast of faded traffic signs and makes important sign details more visible. Gamma correction may be used conditionally for night-time or underexposed images.

4.7 Justification

Contrast stretching is suitable because the system must improve visibility without reversing the natural image appearance. It can be adjusted to enhance the intensity range containing traffic signs.

The selection should be verified using:

- Contrast improvement
- Image quality
- Recognition accuracy
- Processing time
- Preservation of sign details

4.8 Experimental Evaluation

The original image and transformed images should be compared visually and quantitatively.

The experiment should record:

Method	**Contrast**	**Sign Visibility**	**Noise Amplification**	**Recognition Accuracy**
Negative	To be measured	To be measured	To be measured	To be measured
Gamma correction	To be measured	To be measured	To be measured	To be measured
Contrast stretching	To be measured	To be measured	To be measured	To be measured

No numerical performance values should be reported until experiments are completed.

Code:

```
import cv2

import numpy as np
```

```
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Technique 1: Negative Transformation
negative = 255 - gray

# Technique 2: Gamma Correction
gamma = 0.6
normalized = gray / 255.0
gamma_corrected = np.uint8(255 * np.power(normalized, gamma))

# Technique 3: Contrast Stretching
min_value = np.min(gray)
max_value = np.max(gray)

contrast_stretched = np.uint8(
    (gray - min_value) * 255 / (max_value - min_value + 1)
)

# Display results
images = [gray, negative, gamma_corrected, contrast_stretched]
titles = ["Original", "Negative", "Gamma Correction", "Contrast Stretching"]
```

```
plt.figure(figsize=(12, 5))

for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER TASK 2 — HISTOGRAM PROCESSING

5.1 Introduction

A histogram represents the distribution of pixel intensity values in an image. It shows the number of pixels corresponding to each intensity level.

A histogram can help identify whether an image is dark, bright, or has low contrast.

In traffic sign detection, histogram processing is useful for improving the visibility of signs affected by poor lighting, shadows, or faded colors.

5.2 Candidate Technique 1: Global Histogram Equalization

Global Histogram Equalization redistributes image intensity values to improve contrast.

The cumulative distribution function is:

\[CDF(r)=\sum_{i=0}^{r}p(i) \]

The transformed intensity is:

\[s=(L-1)CDF(r) \]

where \(p(i)\) is the normalized histogram.

Advantages

1. Improves global contrast.
2. Simple to implement.
3. Requires relatively low computation.
4. Useful for images with limited intensity range.
5. Can reveal details in low-contrast images.

Limitations

1. May over-enhance some regions.
2. Can increase noise.
3. Does not treat local regions separately.
4. May reduce detail in bright or dark regions.
5. May produce unnatural image appearance.

5.3 Candidate Technique 2: Histogram Matching

Histogram matching modifies an image so that its histogram resembles a target histogram.

Working Principle

A reference image with desirable lighting or contrast is selected. The input image is transformed so that its intensity distribution becomes similar to the reference.

Advantages

1. Can standardize images from different cameras.
2. Useful when a reference image is available.
3. Can improve consistency between image samples.
4. Supports dataset normalization.
5. Allows controlled intensity distribution.

Limitations

1. Requires a suitable reference image.
2. A poor reference may reduce image quality.
3. May not adapt well to every scene.
4. Requires additional processing.
5. May alter important image details.

5.4 Candidate Technique 3: Contrast Limited Adaptive Histogram Equalization

CLAHE divides an image into small regions and applies histogram equalization locally. It limits the contrast amplification to reduce excessive enhancement.

Working Principle

1. Divide the image into tiles.
2. Calculate the histogram of each tile.
3. Limit excessive histogram peaks.
4. Redistribute clipped histogram values.
5. Apply local histogram equalization.
6. Interpolate between neighboring tiles to reduce visible boundaries.

Advantages

1. Improves local contrast.
2. Handles uneven illumination.
3. Enhances faded sign details.
4. Limits excessive contrast amplification.
5. Useful for images containing shadows.

Limitations

1. Requires selection of tile size.
2. Requires selection of clip limit.
3. May amplify noise if parameters are unsuitable.
4. Can increase processing time.
5. May produce artificial local contrast.

5.5 Comparison

Technique	Global or Local	Strength	Limitation
Global histogram equalization	Global	Improves overall contrast	May over-enhance
Histogram matching	Reference-based	Standardizes image appearance	Requires reference
CLAHE	Local	Handles uneven illumination	Requires parameter tuning

5.6 Selected Technique

CLAHE is selected as the primary histogram-processing method.

Traffic signs may be affected by shadows and uneven illumination. CLAHE improves local contrast while limiting excessive amplification.

5.7 Justification

CLAHE is appropriate because a traffic sign may contain important details in a small region of the image. Global histogram equalization may improve the overall image but may not sufficiently enhance the sign region.

CLAHE can improve local visibility while preserving surrounding image information.

5.8 Experimental Evaluation

The methods should be evaluated using:

- Local contrast
- Sign visibility
- Image quality
- Recognition accuracy
- Noise amplification
- Processing time

The clip limit and tile size should be varied during testing. The selected parameters should be documented.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Technique 1: Global Histogram Equalization
global_equalized = cv2.equalizeHist(gray)

# Technique 2: Histogram Matching
# Create a reference image with desired intensity distribution
reference = np.full(gray.shape, 128, dtype=np.uint8)
reference[:, :reference.shape[1] // 2] = 70
reference[:, reference.shape[1] // 2:] = 190

def histogram_matching(source, reference):
```

```
    source_hist, _ = np.histogram(source.flatten(), 256, [0, 256])
    reference_hist, _ = np.histogram(reference.flatten(), 256, [0, 256])

    source_cdf = source_hist.cumsum()
    reference_cdf = reference_hist.cumsum()

    source_cdf = source_cdf / source_cdf[-1]
    reference_cdf = reference_cdf / reference_cdf[-1]

    mapping = np.interp(source_cdf, reference_cdf, np.arange(256))
    matched = mapping[source]

    return np.uint8(matched)

histogram_matched = histogram_matching(gray, reference)

# Technique 3: CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_result = clahe.apply(gray)

# Display results
images = [gray, global_equalized, histogram_matched, clahe_result]
titles = ["Original", "Global Equalization", "Histogram Matching", "CLAHE"]

plt.figure(figsize=(12, 5))

for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER 6: TASK 3 — SPATIAL FILTERING

6.1 Introduction

Spatial filtering modifies pixel values using neighboring pixels. It is commonly used to reduce noise and improve image quality.

Traffic camera images may contain:

- Sensor noise
- Rain droplets
- Random bright or dark pixels
- Small image artifacts
- Low-level noise caused by poor illumination

Spatial filters help reduce these disturbances before edge detection and segmentation.

6.2 Candidate Technique 1: Mean or Box Filter

The mean filter replaces each pixel with the average value of its neighboring pixels.

For a 3×3 neighborhood:

$$ g(x,y)=\frac{1}{9}\sum_{i=-1}^{1}\sum_{j=-1}^{1}f(x+i,y+j) $$

Advantages

1. Simple to implement.
2. Reduces random noise.
3. Requires low computation.
4. Useful for smooth regions.
5. Can be applied to grayscale and color images.

Limitations

1. Blurs edges.
2. Reduces small details.
3. May weaken sign boundaries.
4. Not effective against all impulse noise.
5. May reduce recognition accuracy if overused.

6.3 Candidate Technique 2: Gaussian Filter

The Gaussian filter uses a weighted average of neighboring pixels. Pixels closer to the center receive higher weights.

The Gaussian function is:

$$ G(x,y)=\frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} $$

Advantages

1. Reduces Gaussian noise.
2. Produces smooth image output.
3. Preserves broad structures better than a simple box filter.
4. Useful before edge detection.
5. Can be adjusted using the standard deviation.

Limitations

1. Blurs edges.
2. May remove small sign details.
3. Does not remove impulse noise as effectively as a median filter.
4. Requires parameter selection.
5. Large kernels increase computation.

6.4 Candidate Technique 3: Median Filter

The median filter replaces a pixel with the median value of its neighborhood.

For a neighborhood N:

$$ g(x,y)=Median\{f(i,j):(i,j)\in N\} $$

Advantages

1. Effective against salt-and-pepper noise.
2. Preserves edges better than mean filtering.
3. Removes isolated bright and dark pixels.
4. Useful for rain-related image artifacts.
5. Simple to implement.

Limitations

1. Large kernels may remove small details.
2. May not remove all forms of noise.
3. Requires selection of kernel size.
4. May be computationally slower than a simple mean filter.
5. Strong filtering can alter thin structures.

6.5 Comparison

Technique	Noise Type	Edge Preservation	Main Limitation
Mean filter	Random noise	Low	Blurs edges
Gaussian filter	Gaussian noise	Moderate	Blurs fine details

Median filter	Impulse noise	High	May remove small structures

6.6 Selected Technique

A small median filter is selected for impulse noise and rain-related artifacts.

A Gaussian filter may be used when the image contains mainly sensor noise.

6.7 Justification

Traffic sign boundaries are important for detection. A mean filter may blur these boundaries. A median filter is more suitable for removing isolated bright and dark pixels while preserving edges.

A small kernel such as \(3 \times 3\) may be used initially. The kernel size should be adjusted based on noise severity and sign size.

6.8 Experimental Evaluation

The original and filtered images should be compared using:

- Peak Signal-to-Noise Ratio
- Structural Similarity Index
- Edge preservation
- Recognition accuracy
- Processing time

The experiment should ensure that noise reduction does not remove important sign boundaries.

Code:

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Technique 1: Mean Filter
mean_filter = cv2.blur(gray, (3, 3))

# Technique 2: Gaussian Filter
```

```
gaussian_filter = cv2.GaussianBlur(gray, (5, 5), 0)

# Technique 3: Median Filter
median_filter = cv2.medianBlur(gray, 3)

# Display results
images = [gray, mean_filter, gaussian_filter, median_filter]
titles = ["Original", "Mean Filter", "Gaussian Filter", "Median Filter"]

plt.figure(figsize=(12, 5))

for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER 7: TASK 4 — IMAGE SHARPENING

7.1 Introduction

Image sharpening improves the visibility of edges and fine details. It is used after noise reduction to make important structures more distinct.

In traffic sign detection, sharpening may improve:

- Sign boundaries
- Printed symbols
- Numbers inside speed limit signs
- Shape details
- Small text regions

However, excessive sharpening may amplify noise and create artificial edges.

7.2 Candidate Technique 1: Laplacian Sharpening

The Laplacian is a second-order derivative operator.

$$ \nabla^2f= \frac{\partial^2f}{\partial x^2} + \frac{\partial^2f}{\partial y^2} $$

A sharpened image can be generated using:

$$ g(x,y)=f(x,y)-\lambda\nabla^2f(x,y) $$

where λ controls sharpening strength.

Advantages

1. Enhances fine details.
2. Detects rapid intensity changes.
3. Simple to implement.
4. Useful for sign boundaries.
5. Can improve local edge visibility.

Limitations

1. Amplifies noise.
2. May create artificial edges.
3. Sensitive to small intensity changes.
4. Can produce over-sharpening.
5. Requires parameter selection.

7.3 Candidate Technique 2: Unsharp Masking

Unsharp masking creates a blurred version of the image and subtracts it from the original image.

$$ Mask=f-Blur(f) $$

The sharpened image is:

$$ g=f+kMask $$

where k controls the sharpening amount.

Working Principle

1. Blur the original image.
2. Subtract the blurred image from the original.
3. Obtain a detail mask.
4. Add the mask to the original image.
5. Adjust the sharpening strength.

Advantages

1. Provides controlled sharpening.
2. Enhances sign boundaries.
3. Preserves the overall image appearance.
4. Can be adjusted for different images.
5. Widely used for image enhancement.

Limitations

1. Excessive sharpening creates halos.
2. Noise may be amplified.
3. Requires selection of blur strength.
4. May affect small text.
5. Does not recover information lost through severe blur.

7.4 Candidate Technique 3: Gradient-Based Sharpening

Gradient operators estimate intensity changes in the horizontal and vertical directions.

Common operators include:

- Sobel
- Prewitt
- Scharr

The gradient magnitude is:

$$ |\nabla f|=\sqrt{G_x^2+G_y^2} $$

Advantages

1. Highlights edges.
2. Useful for boundary detection.
3. Provides directional information.
4. Can support segmentation.

5. Helps identify sign shapes.

Limitations

1. May be sensitive to noise.
2. Does not directly restore blurred details.
3. May produce incomplete edges.
4. Requires threshold selection.
5. Can detect background edges as well.

7.5 Comparison

Technique	Main Purpose	Advantage	Limitation
Laplacian	Fine detail enhancement	Strong edge response	Sensitive to noise
Unsharp masking	Controlled sharpening	Adjustable enhancement	May create halos
Gradient sharpening	Edge emphasis	Useful for boundary analysis	May produce incomplete edges

7.6 Selected Technique

Moderate unsharp masking is selected as the primary sharpening method.

It provides controlled enhancement while maintaining the natural appearance of the traffic image.

7.7 Justification

Traffic signs contain important boundaries and internal symbols. Moderate unsharp masking improves these details without applying an excessively strong second-order response.

Sharpening should be performed after noise reduction to avoid amplifying unwanted noise.

7.8 Experimental Evaluation

The sharpening methods should be compared using:

- Edge strength
- Boundary clarity
- Image quality
- Recognition accuracy
- Noise amplification
- Processing time

The selected sharpening amount should be recorded.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Technique 1: Laplacian Sharpening
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian_sharpened = np.uint8(
    np.clip(gray - laplacian, 0, 255)
)

# Technique 2: Unsharp Masking
blurred = cv2.GaussianBlur(gray, (5, 5), 1.0)
unsharp = cv2.addWeighted(gray, 1.5, blurred, -0.5, 0)

# Technique 3: Gradient-Based Sharpening
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

gradient = cv2.magnitude(sobel_x, sobel_y)
gradient = cv2.normalize(gradient, None, 0, 255, cv2.NORM_MINMAX)
gradient = np.uint8(gradient)

# Display results
images = [gray, laplacian_sharpened, unsharp, gradient]
```

```
titles = ["Original", "Laplacian", "Unsharp Masking", "Gradient"]

plt.figure(figsize=(12, 5))

for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER 8: TASK 5 — FREQUENCY-DOMAIN LOW-PASS FILTERING

8.1 Introduction

Frequency-domain filtering processes an image based on its spatial frequency components.

Low-frequency components represent slowly changing image intensity. High-frequency components represent rapid intensity changes, including edges, fine details, and noise.

A low-pass filter allows low-frequency components to pass while reducing high-frequency components.

Low-pass filtering is useful for reducing high-frequency noise in traffic camera images.

8.2 Fourier Transform

The two-dimensional Fourier Transform is:

$$ F(u,v)=\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi(\frac{ux}{M}+\frac{vy}{N})} $$

The filtered image is:

$$ G(u,v)=H(u,v)F(u,v) $$

The output image is obtained using the inverse Fourier Transform.

8.3 Candidate Technique 1: Ideal Low-Pass Filter

The ideal low-pass filter is defined as:

$$ H(u,v)= \begin{cases} 1, & D(u,v)\leq D_0\\ 0, & D(u,v)>D_0 \end{cases} $$

where D_0 is the cutoff frequency.

Advantages

1. Simple frequency response.
2. Removes high-frequency components.
3. Easy to understand.
4. Useful for basic filtering experiments.
5. Provides a clear cutoff frequency.

Limitations

1. May produce ringing artifacts.
2. Can blur sign boundaries.
3. Abrupt frequency transitions may create unwanted effects.
4. Requires cutoff selection.
5. May remove useful fine details.

8.4 Candidate Technique 2: Butterworth Low-Pass Filter

The Butterworth low-pass filter is:

$$ H(u,v)=\frac{1}{1+\left(\frac{D(u,v)}{D_0}\right)^{2n}} $$

where:

- D_0 is the cutoff frequency
- n is the filter order

Advantages

1. Smooth frequency transition.
2. More controlled than the ideal filter.
3. Reduces ringing compared with an ideal cutoff.
4. Allows adjustment of filter order.
5. Useful for frequency-domain experiments.

Limitations

1. Requires selection of cutoff and order.
2. May blur fine details.
3. High order may increase ringing.
4. Requires Fourier-domain computation.
5. May not be necessary for simple images.

8.5 Candidate Technique 3: Gaussian Low-Pass Filter

The Gaussian low-pass filter is:

$$ H(u,v)=e^{-\frac{D^2(u,v)}{2D_0^2}} $$

Advantages

1. Smooth frequency response.
2. Reduces ringing artifacts.
3. Produces natural smoothing.
4. Useful for Gaussian noise reduction.
5. Can be adjusted using the cutoff frequency.

Limitations

1. Blurs sign boundaries.
2. May remove small symbols.
3. Requires cutoff selection.
4. May reduce edge sharpness.
5. Does not remove all types of noise.

8.6 Comparison

Technique	Frequency Transition	Main Advantage	Main Limitation
Ideal LPF	Abrupt	Simple cutoff	Ringing

Butterworth LPF	Adjustable smoothness	Flexible response	Parameter selection
Gaussian LPF	Smooth	Reduced ringing	Blurs details

8.7 Selected Technique

Gaussian low-pass filtering is selected for frequency-domain smoothing.

It provides a smooth frequency response and reduces ringing artifacts.

8.8 Justification

Traffic sign boundaries must remain visible. An ideal low-pass filter may create ringing near strong edges. Gaussian filtering provides smoother suppression of high-frequency noise.

The cutoff frequency must be selected carefully to avoid removing important sign details.

8.9 Experimental Evaluation

The filters should be evaluated using:

- Noise reduction
- Boundary preservation
- PSNR
- SSIM
- Recognition accuracy
- Processing time

The cutoff frequency should be varied and documented.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Fourier Transform
rows, cols = gray.shape
```

```
crow, ccol = rows // 2, cols // 2

dft = np.fft.fft2(gray)
dft_shift = np.fft.fftshift(dft)

# Distance matrix
y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol) ** 2 + (y - crow) ** 2)

D0 = 30

# Technique 1: Ideal Low-Pass Filter
ideal_mask = np.uint8(distance <= D0)

# Technique 2: Butterworth Low-Pass Filter
order = 2
butterworth_mask = 1 / (1 + (distance / D0) ** (2 * order))

# Technique 3: Gaussian Low-Pass Filter
gaussian_mask = np.exp(-(distance ** 2) / (2 * D0 ** 2))

def apply_filter(mask):
    filtered = dft_shift * mask
    inverse = np.fft.ifftshift(filtered)
    result = np.fft.ifft2(inverse)
    result = np.abs(result)
    return np.uint8(np.clip(result, 0, 255))

ideal_result = apply_filter(ideal_mask)
butterworth_result = apply_filter(butterworth_mask)
gaussian_result = apply_filter(gaussian_mask)

# Display results
```

```
images = [gray, ideal_result, butterworth_result, gaussian_result]
titles = ["Original", "Ideal LPF", "Butterworth LPF", "Gaussian LPF"]

plt.figure(figsize=(12, 5))

for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER 9: TASK 6 — FREQUENCY-DOMAIN HIGH-PASS FILTERING

9.1 Introduction

A high-pass filter reduces low-frequency components and emphasizes high-frequency components. High-frequency components include edges, boundaries, and fine details.

In traffic sign detection, high-pass filtering can support boundary enhancement and shadow-related image analysis.

However, high-pass filtering may also amplify noise. Therefore, it should be used carefully.

9.2 Candidate Technique 1: Ideal High-Pass Filter

The ideal high-pass filter is:

$$ H(u,v)= \begin{cases} 0, & D(u,v)\leq D_0\\ 1, & D(u,v)>D_0 \end{cases} $$

Advantages

1. Removes low-frequency components.
2. Emphasizes sharp intensity changes.
3. Simple to implement.
4. Useful for studying high-frequency information.
5. Can highlight sign boundaries.

Limitations

1. May produce ringing artifacts.
2. Amplifies noise.
3. May create artificial edges.
4. Requires cutoff selection.
5. May remove useful background information.

9.3 Candidate Technique 2: High-Frequency Emphasis Filtering

High-frequency emphasis filtering combines the original low-frequency information with enhanced high-frequency components.

$$ H_{HFE}(u,v)=a+bH_{HP}(u,v) $$

where:

- a controls the low-frequency contribution
- b controls high-frequency emphasis

Advantages

1. Retains some original image information.
2. Enhances fine details.
3. Can improve boundary visibility.
4. More flexible than pure high-pass filtering.

5. Useful for sharpening.

Limitations

1. Requires parameter tuning.
2. May amplify noise.
3. Can create excessive contrast.
4. May produce unnatural appearance.
5. Requires frequency-domain processing.

9.4 Candidate Technique 3: Homomorphic Filtering

Homomorphic filtering separates illumination and reflectance components.

The image is modeled as:

$$ f(x,y)=i(x,y)r(x,y) $$

Taking the logarithm:

$$ \ln f(x,y)=\ln i(x,y)+\ln r(x,y) $$

The illumination component is generally associated with lower frequencies, while reflectance contains more detailed information.

Advantages

1. Can reduce uneven illumination.
2. May improve contrast under shadows.
3. Enhances details in poorly illuminated regions.
4. Useful for images with bright and dark areas.
5. Can support night-time image enhancement.

Limitations

1. Requires parameter tuning.
2. May amplify noise.
3. Can alter natural colors.
4. May produce halos.
5. Does not guarantee recovery of severely underexposed information.

9.5 Comparison

Technique	Main Purpose	Advantage	Limitation
Ideal HPF	High-frequency extraction	Strong edge emphasis	Ringing and noise
High-frequency emphasis	Detail enhancement	Retains original information	Parameter tuning

Homomorphic filtering	Illumination correction	Handles uneven lighting	May alter appearance

9.6 Selected Technique

Homomorphic filtering is selected conditionally for images affected by shadows or uneven illumination.

It is not required for every image. High-frequency emphasis may be used for detail enhancement when needed.

9.7 Justification

Traffic images may contain strong illumination variations. Homomorphic filtering can reduce the influence of uneven illumination and improve the visibility of sign details.

The output should be checked to ensure that the enhancement does not create false edges or excessive noise.

9.8 Experimental Evaluation

The methods should be evaluated using:

- Illumination uniformity
- Sign visibility
- Edge preservation
- Recognition accuracy
- Image quality
- Processing time

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Technique 1: Simple Region Growing
```

```
# Select a seed point near the traffic sign
seed_x = gray.shape[1] // 2
seed_y = gray.shape[0] // 2

seed_value = int(gray[seed_y, seed_x])
threshold_value = 20

region_growing = np.zeros_like(gray)

for y in range(gray.shape[0]):
    for x in range(gray.shape[1]):
        if abs(int(gray[y, x]) - seed_value) < threshold_value:
            region_growing[y, x] = 255

# Technique 2: Watershed
blurred = cv2.GaussianBlur(image, (5, 5), 0)
gray_blurred = cv2.cvtColor(blurred, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(
    gray_blurred, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)

sure_background = cv2.dilate(opening, kernel, iterations=3)

distance_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_foreground = cv2.threshold(
    distance_transform,
    0.5 * distance_transform.max(),
    255,
```

```
    0
)

sure_foreground = np.uint8(sure_foreground)
unknown = cv2.subtract(sure_background, sure_foreground)

_, markers = cv2.connectedComponents(sure_foreground)
markers = markers + 1
markers[unknown == 255] = 0

watershed_result = image.copy()
cv2.watershed(watershed_result, markers)
watershed_result[markers == -1] = [0, 0, 255]
watershed_result = cv2.cvtColor(watershed_result, cv2.COLOR_BGR2RGB)

# Technique 3: Edge Linking using Contours
edges = cv2.Canny(gray, 50, 150)
contours, _ = cv2.findContours(
    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

edge_linking = image.copy()
cv2.drawContours(edge_linking, contours, -1, (0, 255, 0), 2)
edge_linking = cv2.cvtColor(edge_linking, cv2.COLOR_BGR2RGB)

# Display results
images = [gray, region_growing, watershed_result, edge_linking]
titles = ["Original", "Region Growing", "Watershed", "Edge Linking"]

plt.figure(figsize=(12, 5))

for i in range(4):
    plt.subplot(1, 4, i + 1)
```

```
    if len(images[i].shape) == 2:
        plt.imshow(images[i], cmap="gray")
    else:
        plt.imshow(images[i])

    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER 10: TASK 7 — POINT, LINE, AND EDGE DETECTION

10.1 Introduction

Point, line, and edge detection are important techniques for identifying significant structures in an image.

Traffic signs often contain geometric boundaries and internal symbols. Detecting these structures can help locate candidate sign regions.

10.2 Candidate Technique 1: Point Detection

Point detection identifies isolated pixels that differ significantly from their surrounding neighborhood.

A point mask may be represented as:

$$\begin{bmatrix} -1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1 \end{bmatrix}$$

The response is calculated by applying the mask to the image. A large response may indicate an isolated point.

Advantages

1. Detects isolated intensity changes.
2. Simple to implement.
3. Useful for identifying small visual features.
4. Can support noise analysis.
5. Requires low computational complexity.

Limitations

1. Sensitive to noise.
2. May detect unwanted isolated pixels.
3. Does not identify complete sign boundaries.
4. Requires threshold selection.
5. Limited usefulness as the main traffic sign detector.

10.3 Candidate Technique 2: Line Detection

Line detection identifies linear structures in an image. It can be performed using directional masks or the Hough Transform.

The Hough line representation is:

$$\rho=x\cos\theta+y\sin\theta$$

where ρ is the distance from the origin and θ is the line angle.

Advantages

1. Detects straight boundaries.
2. Useful for geometric structures.
3. Can identify road markings.

4. Supports shape analysis.
5. Can detect lines even when partly interrupted.

Limitations

1. May detect background lines.
2. Requires parameter selection.
3. Less suitable for curved boundaries.
4. Can be affected by noise.
5. May produce multiple detections for one structure.

10.4 Candidate Technique 3: Canny Edge Detection

Canny edge detection identifies strong image boundaries while reducing noise sensitivity.

The major steps are:

1. Gaussian smoothing
2. Gradient calculation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking by hysteresis

Advantages

1. Produces thin edges.
2. Reduces noise sensitivity through smoothing.
3. Detects strong boundaries.
4. Supports segmentation.
5. Works well for many geometric shapes.

Limitations

1. Requires threshold selection.
2. May produce broken edges.
3. Can detect background structures.
4. May fail under severe blur.
5. Requires additional processing for complete sign regions.

10.5 Comparison

Technique	Detects	Main Advantage	Main Limitation
Point detection	Isolated points	Finds local intensity changes	Limited boundary information

Line detection	Straight lines	Detects linear structures	May detect background lines
Canny	General edges	Thin and relatively robust edges	Threshold-dependent

10.6 Selected Technique

Canny edge detection is selected as the primary edge detection method.

Point and line detection may be used as supporting methods when the application requires isolated-feature or straight-line analysis.

10.7 Justification

Traffic signs have boundaries that may be circular, triangular, rectangular, or polygonal. Canny can identify general boundaries rather than only isolated points or straight lines.

The output of Canny can be used for contour extraction and region segmentation.

10.8 Experimental Evaluation

The methods should be compared using:

- Edge detection accuracy
- Boundary completeness
- False edge rate
- Processing time
- Recognition accuracy

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("traffic_sign.jpg")

if image is None:
    print("Image not found")
    exit()

# Step 1: Resize image
image = cv2.resize(image, (640, 480))
```

```
# Step 2: Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Apply CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
enhanced = clahe.apply(gray)

# Step 4: Reduce noise
denoised = cv2.medianBlur(enhanced, 3)

# Step 5: Sharpen using unsharp masking
blurred = cv2.GaussianBlur(denoised, (5, 5), 1.0)
sharpened = cv2.addWeighted(denoised, 1.5, blurred, -0.5, 0)

# Step 6: Detect edges
edges = cv2.Canny(sharpened, 50, 150)

# Display processing stages
images = [gray, enhanced, denoised, sharpened, edges]
titles = ["Grayscale", "CLAHE", "Denoised", "Sharpened", "Canny Edges"]

plt.figure(figsize=(15, 4))

for i in range(len(images)):
    plt.subplot(1, len(images), i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

CHAPTER 11: TASK 8 — THRESHOLDING

11.1 Introduction

Thresholding converts an image into a binary representation based on intensity values. It separates pixels into foreground and background categories.

For traffic sign detection, thresholding can help isolate candidate sign regions based on brightness, color, or intensity.

11.2 Candidate Technique 1: Global Thresholding

Global thresholding applies a single threshold value to the entire image.

\[g(x,y)= \begin{cases} 1, & f(x,y)>T\\ 0, & f(x,y)\leq T \end{cases} \]

Advantages

1. Simple to implement.
2. Requires low computation.
3. Useful under uniform illumination.
4. Produces a binary image.
5. Easy to interpret.

Limitations

1. Sensitive to lighting changes.
2. Performs poorly under shadows.
3. Requires threshold selection.
4. May fail for faded signs.
5. May separate the sign incorrectly from the background.

11.3 Candidate Technique 2: Otsu Thresholding

Otsu thresholding automatically selects a threshold by maximizing the separation between two intensity classes.

The method uses the image histogram to identify a threshold that separates foreground and background pixels.

Advantages

1. Automatically selects a threshold.
2. Does not require manual threshold selection.
3. Works well for bimodal histograms.
4. Simple to implement.
5. Useful for images with clear foreground-background separation.

Limitations

1. May fail when the histogram is not bimodal.
2. Sensitive to uneven illumination.
3. May be affected by noise.
4. May not separate faded signs.
5. Does not directly identify traffic sign categories.

11.4 Candidate Technique 3: Adaptive Thresholding

Adaptive thresholding calculates a local threshold for each pixel based on its neighborhood.

\[T(x,y)=Mean(N_{x,y})-C \]

where \(N_{x,y}\) is the local neighborhood and \(C\) is a constant.

Advantages

1. Handles uneven illumination.
2. Useful for images containing shadows.
3. Supports local foreground-background separation.
4. Can improve faded region visibility.
5. Does not depend on one global threshold.

Limitations

1. Requires neighborhood size selection.
2. May produce noisy binary images.
3. Can create inconsistent regions.
4. Requires parameter tuning.
5. May be computationally more expensive than global thresholding.

11.5 Comparison

Technique	Threshold Type	Advantage	Limitation
Global	Single threshold	Simple and fast	Sensitive to illumination
Otsu	Automatically selected global threshold	Reduces manual selection	Depends on histogram structure
Adaptive	Local threshold	Handles uneven lighting	Requires neighborhood parameters

11.6 Selected Technique

Adaptive thresholding is selected for images with uneven illumination.

Otsu thresholding may be used when the image has a clear foreground-background histogram.

11.7 Justification

Traffic signs may appear differently across the same image due to shadows, sunlight, and reflections. Adaptive thresholding calculates local thresholds and is therefore more suitable for uneven illumination.

11.8 Experimental Evaluation

Thresholding methods should be evaluated using:

- Segmentation accuracy
- Foreground-background separation
- Boundary preservation
- False positive regions
- Processing time

CHAPTER 12: TASK 9 — REGION SEGMENTATION

12.1 Introduction

Region segmentation divides an image into meaningful regions. It is used to separate traffic signs from the background and identify candidate objects for recognition.

Traffic sign segmentation is challenging because signs may have similar colors to nearby objects, may be partially hidden, or may be affected by poor lighting.

12.2 Candidate Technique 1: Region Growing

Region growing begins with one or more seed pixels and adds neighboring pixels that satisfy a similarity condition.

A simple similarity condition is:

$$ |I(p)-I(seed)|<T $$

where T is a threshold.

Working Principle

1. Select seed pixels.
2. Examine neighboring pixels.
3. Compare their intensity or color with the region.
4. Add pixels that satisfy the similarity condition.
5. Continue until no more pixels meet the condition.

Advantages

1. Produces connected regions.
2. Uses local similarity.
3. Can identify sign regions.
4. Simple to understand.
5. Supports segmentation of relatively uniform regions.

Limitations

1. Depends on seed selection.
2. Sensitive to threshold selection.
3. May leak into the background.
4. Can fail under uneven illumination.
5. May not handle complex sign backgrounds.

12.3 Candidate Technique 2: Watershed Segmentation

Watershed treats an image as a topographic surface. Regions are formed based on boundaries and intensity gradients.

Working Principle

1. Calculate image gradients.
2. Identify markers or seed regions.
3. Simulate flooding from the markers.
4. Form boundaries where regions meet.
5. Obtain segmented regions.

Advantages

1. Separates touching objects.
2. Produces clear region boundaries.
3. Useful for complex segmentation.
4. Can use marker information.
5. Supports object separation.

Limitations

1. May produce over-segmentation.
2. Requires suitable markers.
3. Sensitive to noise.
4. May create too many regions.
5. Requires additional post-processing.

12.4 Candidate Technique 3: Edge Linking

Edge linking connects edge pixels to form meaningful boundaries.

Working Principle

1. Detect edges.
2. Identify nearby edge pixels.
3. Compare direction and intensity.
4. Connect compatible edge segments.
5. Form candidate object boundaries.

Advantages

1. Supports boundary completion.
2. Useful when edges are broken.
3. Can help identify sign outlines.
4. Uses geometric information.
5. Can support contour extraction.

Limitations

1. Depends on edge quality.
2. May connect unrelated edges.
3. Requires linking parameters.
4. May fail under severe occlusion.
5. Can produce incomplete boundaries.

12.5 Comparison

Technique	Main Principle	Advantage	Limitation
Region growing	Pixel similarity	Connected regions	Seed-sensitive
Watershed	Topographic boundaries	Separates touching regions	Over-segmentation
Edge linking	Connects edge segments	Completes boundaries	Depends on edge quality

12.6 Selected Technique

Region growing combined with edge linking is selected as the primary segmentation strategy.

Watershed may be used when multiple touching objects need to be separated.

12.7 Justification

Region growing can group pixels that have similar intensity or color. Edge linking can help complete sign boundaries when some edge pixels are missing.

The combination can support candidate traffic sign extraction. However, partial occlusion and complex backgrounds may require a trained object detector.

12.8 Experimental Evaluation

Segmentation should be evaluated using:

- Intersection over Union
- Dice coefficient
- Boundary error
- Region completeness
- False segmentation rate
- Processing time

CHAPTER 13: TASK 10 — INTEGRATED TRAFFIC SIGN DETECTION STRATEGY

13.1 Introduction

The integrated system combines the image-processing techniques selected in the previous tasks with a traffic sign recognition model.

The purpose of the integrated system is to create a complete pipeline that can process road images and identify traffic signs under different environmental conditions.

13.2 Candidate Technique 1: Traditional Computer Vision

Traditional computer vision uses manually designed rules and image-processing operations.

Typical methods include:

- Color thresholding
- Shape detection
- Contour extraction
- Edge detection
- Template matching
- Feature descriptors

Advantages

1. Requires relatively low computational resources.
2. Can be implemented using standard image-processing libraries.
3. Provides interpretable processing steps.
4. Useful for controlled environments.
5. Does not always require large training datasets.

Limitations

1. Sensitive to lighting conditions.
2. Requires manual parameter tuning.
3. May fail under occlusion.
4. May confuse similar objects.
5. Less flexible for complex backgrounds.

13.3 Candidate Technique 2: Deep Learning-Based Detection

Deep learning-based detection uses a trained model to identify traffic signs.

The model learns visual features from training images. It can detect objects and classify their categories.

Advantages

1. Learns complex visual features.

2. Can handle varied backgrounds.
3. May improve recognition under partial occlusion.
4. Supports multiple sign categories.
5. Can be trained using labeled datasets.

Limitations

1. Requires labeled training data.
2. Training may require substantial computation.
3. Performance depends on dataset quality.
4. May require GPU resources.
5. Can be difficult to interpret.

13.4 Candidate Technique 3: Hybrid Computer Vision and Deep Learning

The hybrid approach combines traditional image processing with a trained detector.

Traditional methods improve image quality and support candidate region extraction. The trained model performs final recognition.

Advantages

1. Combines enhancement and learned recognition.
2. Supports difficult lighting conditions.
3. Can reduce irrelevant image information.
4. Provides flexibility in processing.
5. Can support multiple traffic sign categories.

Limitations

1. Requires careful integration.
2. Additional preprocessing may increase latency.
3. Enhancement may sometimes reduce model performance.
4. Requires parameter tuning.
5. Needs experimental verification.

13.5 Comparison

Approach	Strength	Limitation
Traditional computer vision	Simple and interpretable	Sensitive to environmental changes
Deep learning	Learns complex features	Requires training data
Hybrid approach	Combines enhancement and recognition	Requires integration and testing

13.6 Selected Technique

A hybrid computer vision and deep learning approach is selected.

The image-processing stages improve the input image, while the trained detector identifies traffic sign categories.

13.7 Justification

The traffic sign detection problem includes changing illumination, rain, faded signs, occlusion, and motion blur. Traditional methods alone may not be sufficiently robust for all conditions.

A hybrid approach allows image enhancement and noise reduction to support the recognition model. The final decision must be based on experimental results because preprocessing may improve or reduce recognition accuracy depending on the model and dataset.

13.8 Integrated Processing Sequence

The proposed processing sequence is:

1. Capture the road image.
2. Resize the image if required.
3. Convert the image to a suitable color space.
4. Apply contrast enhancement.
5. Apply local histogram processing.
6. Reduce noise.
7. Apply moderate sharpening.
8. Use frequency-domain filtering when necessary.
9. Detect edges and boundaries.
10. Apply thresholding.
11. Segment candidate sign regions.
12. Pass the image or candidate regions to the detector.
13. Classify traffic signs.
14. Display bounding boxes and labels.
15. Store results for evaluation.

13.9 Experimental Evaluation

The integrated system should be evaluated using:

- Detection accuracy
- Precision
- Recall
- F1-score
- Intersection over Union
- False positive rate

- False negative rate
- Processing time
- Frames per second

The system should be tested separately under sunlight, night-time, rain, shadows, faded signs, occlusion, and motion blur.

CHAPTER 14: PERFORMANCE EVALUATION

14.1 Introduction

Performance evaluation is necessary to determine whether the selected techniques improve traffic sign detection.

The evaluation should compare the original image with the processed image and compare recognition results before and after preprocessing.

14.2 Peak Signal-to-Noise Ratio

PSNR measures the difference between an original image and a processed image.

$$PSNR=10\log_{10}\left(\frac{MAX_I^2}{MSE}\right)$$

where:

- MAX_I is the maximum possible pixel value
- MSE is the mean squared error

A higher PSNR generally indicates lower pixel-level error. However, PSNR alone does not guarantee better traffic sign recognition.

14.3 Structural Similarity Index

SSIM measures similarity between two images based on luminance, contrast, and structure.

$$SSIM(x,y)= \frac{(2\mu_x\mu_y+C_1)(2\sigma_{xy}+C_2)}{(\mu_x^2+\mu_y^2+C_1)(\sigma_x^2+\sigma_y^2+C_2)}$$

SSIM can help evaluate whether image enhancement preserves important structures.

14.4 Precision

Precision measures the proportion of detected traffic signs that are correct.

$$Precision=\frac{TP}{TP+FP}$$

where:

- TP = True Positives
- FP = False Positives

High precision indicates fewer false detections.

14.5 Recall

Recall measures the proportion of actual traffic signs detected by the system.

$$Recall=\frac{TP}{TP+FN}$$

where FN represents False Negatives.

High recall indicates that the system misses fewer traffic signs.

14.6 F1-Score

The F1-score combines precision and recall.

$$F1=2\frac{Precision\times Recall}{Precision+Recall}$$

It is useful when both false detections and missed signs are important.

14.7 Intersection over Union

IoU measures the overlap between the predicted bounding box and the ground-truth bounding box.

\[IoU=\frac{Area(Prediction\cap GroundTruth)} {Area(Prediction\cup GroundTruth)} \]

Higher IoU indicates better localization.

14.8 Dice Coefficient

The Dice coefficient measures overlap between segmented regions.

\[Dice=\frac{2|A\cap B|}{|A|+|B|} \]

It is useful for evaluating segmentation quality.

14.9 Processing Time

Processing time measures the time required to process one image or frame.

For real-time applications, the system should minimize latency while maintaining acceptable detection accuracy.

14.10 Evaluation Table

Metric	Purpose	Result
PSNR	Pixel-level image quality	To be measured
SSIM	Structural similarity	To be measured
Precision	False detection control	To be measured
Recall	Missed sign control	To be measured
F1-score	Combined detection performance	To be measured
IoU	Bounding-box localization	To be measured
Dice	Segmentation overlap	To be measured
Processing time	Computational efficiency	To be measured

Important: Numerical values should be entered only after performing experiments. Expected performance should not be presented as measured results.

CHAPTER 15: RESULTS AND DISCUSSION

15.1 Introduction

The results and discussion section explains how the proposed techniques affect traffic sign detection.

The system should be tested using images captured under different environmental conditions. The results should be compared with the original images and with alternative processing methods.

15.2 Image Enhancement Results

The grey-level transformation methods should be evaluated based on contrast improvement and sign visibility.

Contrast stretching is expected to improve the visibility of faded signs. Gamma correction may improve dark images. Negative transformation may be useful for visual analysis but is not expected to be the primary enhancement method.

15.3 Histogram Processing Results

Global histogram equalization may improve overall contrast. Histogram matching may standardize images when a suitable reference is available. CLAHE is expected to improve local contrast under uneven illumination.

The selected method should be verified using image quality metrics and recognition performance.

15.4 Spatial Filtering Results

Mean filtering may reduce random noise but can blur boundaries. Gaussian filtering may reduce Gaussian noise. Median filtering is expected to preserve edges while removing impulse noise.

The experiment should determine whether filtering improves recognition or removes important sign details.

15.5 Sharpening Results

Laplacian sharpening may produce strong edge responses. Unsharp masking provides adjustable sharpening. Gradient-based methods highlight boundaries.

Moderate unsharp masking is expected to improve sign details while reducing excessive sharpening artifacts.

15.6 Frequency Filtering Results

Gaussian low-pass filtering may reduce high-frequency noise with fewer ringing artifacts than ideal filtering. Homomorphic filtering may improve images affected by uneven illumination.

The results should be analyzed to determine whether frequency-domain processing improves recognition enough to justify its computational cost.

15.7 Edge Detection Results

Canny edge detection is expected to identify important boundaries and support segmentation. Point detection may identify isolated features, while line detection may identify straight structures.

The experiment should measure edge completeness and false edge rate.

15.8 Thresholding Results

Global thresholding may work under uniform illumination. Otsu thresholding may automatically identify a suitable threshold for images with clear intensity classes. Adaptive thresholding is expected to perform better under uneven illumination.

15.9 Segmentation Results

Region growing may produce connected regions when the seed and similarity threshold are appropriate. Watershed may separate touching regions. Edge linking may help complete broken boundaries.

The segmentation result should be compared with ground-truth regions using IoU and Dice coefficient.

15.10 Integrated System Results

The final system should compare recognition results under the following conditions:

Condition	Expected Challenge	Evaluation
Sunlight	Overexposure	Detection accuracy
Night	Low illumination	Recall and image quality
Rain	Noise and streaks	Precision and recall
Shadows	Uneven illumination	Segmentation quality
Faded signs	Low contrast	Recognition accuracy
Occlusion	Missing sign details	Detection accuracy
Motion blur	Weak boundaries	Recognition accuracy

The final conclusion should be based on the measured results.

CHAPTER 16: LIMITATIONS

16.1 Environmental Limitations

The system may experience reduced performance under extreme weather conditions, severe darkness, heavy rain, or strong glare.

16.2 Occlusion Limitations

A traffic sign that is heavily covered by another object may not contain sufficient visual information for reliable recognition.

16.3 Motion Blur Limitations

Severe motion blur may remove important sign details. Image enhancement cannot fully recover information that was not captured.

16.4 Dataset Limitations

The recognition model may perform poorly if the training dataset does not contain enough examples of faded signs, night-time images, rain, and partial occlusion.

16.5 Computational Limitations

Some enhancement and deep learning methods may require more processing time than is available for real-time applications.

16.6 Parameter Limitations

Traditional image-processing methods require parameter selection. A parameter that works well for one image may not work well for another image.

CHAPTER 17: FUTURE SCOPE

17.1 Real-Time Video Processing

The system can be extended to process continuous video from traffic cameras. This would allow real-time traffic sign detection.

17.2 Edge Computing

The system can be implemented on edge devices installed near traffic cameras. This may reduce communication delay and cloud processing requirements.

17.3 Advanced Deep Learning Models

Future work may include testing more advanced object detection models to improve accuracy and speed.

17.4 Automatic Image Enhancement

A system can be developed to automatically select enhancement parameters based on image conditions.

17.5 Multi-Frame Recognition

Multiple video frames can be combined to improve recognition when a sign is partially hidden in one frame.

17.6 Damaged Sign Detection

The system can be extended to identify damaged, faded, bent, or obstructed traffic signs and report them to traffic authorities.

17.7 Smart City Integration

The detected traffic sign information can be integrated with smart city dashboards and traffic management systems.

17.8 Driver Assistance

The system can provide warnings to drivers when important signs are detected, such as STOP, No Entry, or speed limit signs.

CHAPTER 18: CONCLUSION

The Smart Traffic Sign Detection System provides a computer-vision framework for detecting and recognizing traffic signs under different environmental conditions.

The project studies the use of grey-level transformations, histogram processing, spatial filtering, sharpening, frequency-domain filtering, point and line detection, edge detection, thresholding, and region segmentation.

Three candidate techniques are considered for each major task. Their advantages and limitations are compared, and suitable techniques are selected according to the requirements of traffic sign detection.

Piecewise-linear contrast stretching is selected for contrast enhancement. CLAHE is selected for local contrast improvement. Median filtering is selected for impulse noise, while Gaussian filtering may be used for sensor noise. Moderate unsharp masking is selected for controlled sharpening. Gaussian low-pass filtering is selected for smooth frequency-domain noise reduction. Homomorphic filtering is used conditionally for uneven illumination. Canny edge detection is selected for general boundary detection. Adaptive thresholding is selected for uneven illumination, and region growing combined with edge linking is selected for candidate region segmentation.

A hybrid approach combining traditional image processing and deep learning is selected for the integrated system. Traditional image processing improves image quality and supports region extraction, while the trained detector performs final traffic sign recognition.

The system can support smart city traffic monitoring, intelligent transportation systems, and driver assistance applications. The final performance must be verified using a suitable dataset and measurable evaluation metrics such as precision, recall, F1-score, IoU, Dice coefficient, PSNR, SSIM, and processing time.

The project demonstrates that a carefully designed image-processing pipeline can support traffic sign recognition. However, the effectiveness of each technique depends on the image conditions, parameter settings, dataset quality, and recognition model. Therefore, experimental evaluation is essential before deploying the system in real-world traffic environments.

MARK DISTRIBUTION

Task	Marks
Task 1 – Grey Level Transformation Analysis	4
Task 2 – Histogram Processing and Enhancement	4
Task 3 – Spatial Filtering and Image Smoothing	5
Task 4 – Image Sharpening and Edge Enhancement	5
Task 5 – Frequency-Domain Low-Pass Filtering	5
Task 6 – Frequency-Domain High-Pass Filtering	4
Task 7 – Point, Line, and Edge Detection	5
Task 8 – Image Thresholding and Binarization	4
Task 9 – Region-Based Segmentation and Integrated Pipeline Design	6
Task 10 – Smart Scanner Adaptation for High-Resolution Phone Photographs	8
TOTAL	50